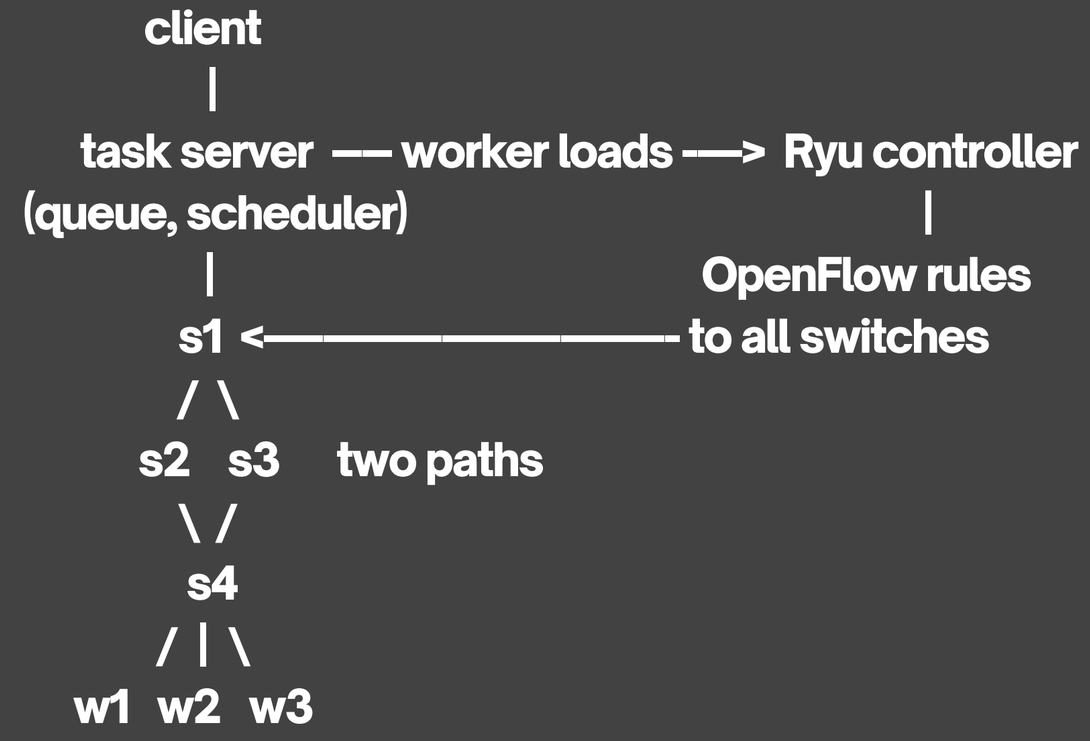
Distributed Task Notification and Worker System

-Saisree, Sanooj Rijesh,
Shirin Sai B

Problem and Goal

- Workers are not equally busy, and taking turns (round robin) makes a busy worker slow down the whole batch.
- Network traffic has the same issue: one path becomes the bottleneck while other links sit idle.
- Goal: balance both compute (which worker) and network (which path).

Architecture and Work division



Client, server and workers talk over TCP.
Server writes loads to /tmp/dtn_worker_loads.json every 2s for Ryu.
Python standard library only, except Mininet and Ryu for SDN.
Person 1: server and scheduler |
Person 2: workers and client |
Person 3: SDN and networking

Person 1: Task Server and Scheduler

Task Server

- TCP server on port 5050; workers and clients both connect.
- Unique id per task, a queue of waiting tasks, and a registry of workers with load and running-task count.
- Stores results sent back by workers.
- Messages are one line of JSON over TCP: REGISTER, TASK, RESULT, STATUS, SUBMIT, ACK, GET_RESULT.

Scheduler

- Least-loaded (default): lowest load among workers with a free slot.
- Round robin (--policy round_robin): baseline for the benchmark.
- Log line example: task 1 (factorial) -> worker2 loads: worker1=69% worker2=12% worker3=42%
- When the best worker fills up, the next one gets tasks.

Fault tolerance and SDN Bridge

- No heartbeat for 10 s, or a disconnect, means the worker has failed. Its unfinished tasks go to the front of the queue for another worker.
- Every 2 s the loads file is written: the link between scheduler and controller.
- Limitations: results in memory only, no socket authentication.

Person 2: Workers and Communication

Worker and tasks

- Registers with a name, receives TASK, runs each task in its own thread, returns RESULT.
- Tasks: factorial, prime_check, sum_range, sort, matrix_multiply. Unknown operations are rejected.

Heartbeat and simulated load

- STATUS heartbeat every 2 s, plus immediately when a task starts or finishes.
- --load fakes background load; busier workers run tasks slower, like a real loaded machine.
- --vary changes load randomly every 15 s.
- Load = background plus a share per running task (simulated, not real CPU).

Resilience and client tool

- If the server goes down, the worker retries every 3 s and re-registers automatically.
- Client CLI: status, submit <op> <input> --wait, demo (10 tasks), demo --repeat N.
- Demo idea: stop the server, show “trying again in 3 seconds”, restart and watch workers rejoin.

Person 3: SDN and Networking

Topology and Ryu controller

- Mininet: 4 switches, server on s1, workers on s4, two parallel paths (via s2 and via s3), 10 Mbit/s links.
- Ryu with OpenFlow 1.3; it answers ARP itself because the loop would make flooding circulate forever.
- Installs IP forwarding rules and reads port stats from s1 to print per-path traffic.

Path planning and link failure

- Every 3 s the controller reads loads, sorts workers, splits them across paths, and reroutes those that changed.
- Example: w1=85%, w2=15%, w3=60% gives w1 alone on s2, w2 and w3 on s3.
- path_planner.py runs without Mininet for a laptop demo.
- Link down: controller moves all workers to the surviving path within a second or two; when it returns, they are split again.
- DTN_ROUTING=static sends everything via s2 (benchmark baseline only).

Demo, Results

01

Flow: start Ryu, then topology, pingall, client demo, dump-flows s1 (output:2 = via s2, output:3 = via s3), restart a worker with new load to see reroute, then link s1 s2 down.

02

Automated tests: correct task results, least-loaded choice, batch spread, task recovery after worker kill, rejected unknown ops, round robin, path planner.

03

SDN check: 14/14 checks passed (switch connect, pingall, path split, reroute, link failure and recovery)

Results, Scheduling

Condition	Scheduler	Total	Avg/task
Uneven 70/10/40	Round robin	10.1 s	5.2 s
Uneven 70/10/40	Least loaded	6.1 s	3.4 s
Equal 30/30/30	Both	6.1 s	3.4 s

About 40% faster with uneven loads (w1=6, w2=16, w3=8 tasks); identical when loads are equal, as expected.

Results, Routing

Tasks	Routing	Total	Packets s2 / s3
6	Static	6.6 s	233 / 0
6	Dynamic	4.8 s	131 / 140
12	Static	10.7 s	375 / 0
12	Dynamic	6.2 s	174 / 202

28% faster with 6 tasks, 42% faster with 12: more traffic, bigger benefit.

Limitations, conclusion

Limitations: in-memory results, no socket security, fixed topology, load-only balancing, simulated load, link handling covers four path links only.
Future work: persistent storage, authentication and encryption, real CPU metrics, use measured traffic in routing.
Conclusion: scheduler (compute) plus SDN controller (network) give about 40% and up to 42% gains, and the system survives worker and link failures.

Thank You